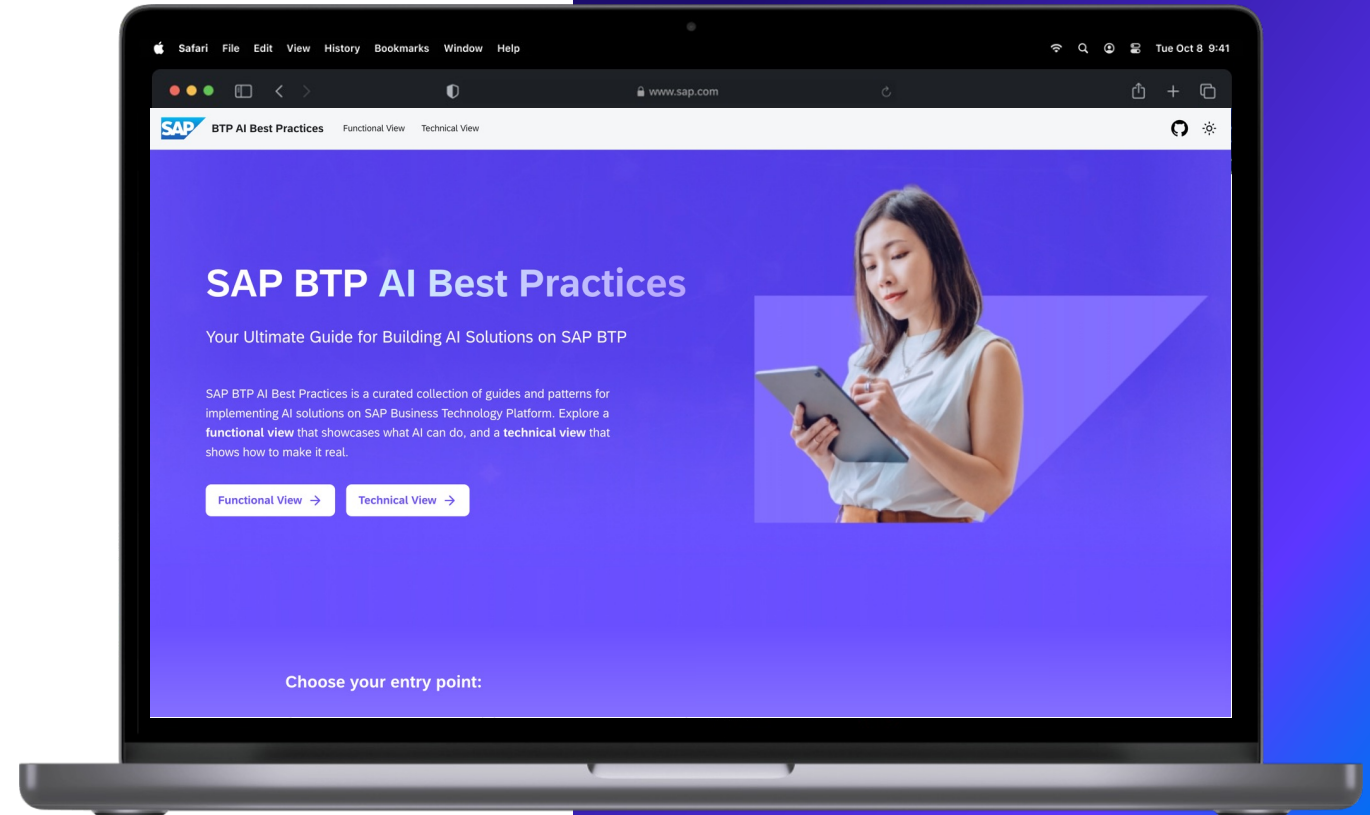


SAP BTP AI Best Practices

Pro-Code AI Agents

Build production-ready SAP Agents with ReAct, LangGraph and governed enterprise tools



BTP AI Services Center of Excellence

25.05.26

Steps

- 1 Overview**
- 2 Pre-requisites**
- 3 Key Choices and Guidelines**
- 4 Implementation**

Why Pro-Code Agents?

Use code when the workflow needs custom control, integrations, and reliability

1

Joule scenarios

Best for built-in SAP conversational capabilities and guided business interactions.

Use when the standard Joule experience already fits the process.

2

Content-based agents

Best for grounded answers and actions based on approved business content.

Use when the main challenge is enterprise knowledge and declarative orchestration.

3

Pro-code agents

Best for custom workflows, APIs, state, algorithms, and advanced control.

Use when the agent must plan, call tools, branch, validate, and be debugged like software.

Expected Outcomes and Benefits

What should a well-designed pro-code agent deliver?

Expected outcome

- Understand how agents reason and act through the ReAct paradigm
- Design single-agent and multi-step workflows with LangGraph
- Integrate SAP services, external APIs, and enterprise data sources
- Make informed choices about tools, state, routing, and controls

1

Autonomous reasoning

Break complex requests into controlled steps

2

Dynamic tools

Select APIs, databases, or calculations based on context

3

Workflow control

Model branching, gates, validation, and approvals

4

Debuggability

Inspect state, tool calls, traces, and decisions

Agentic Patterns at a Glance

What should a well-designed pro-code agent deliver?

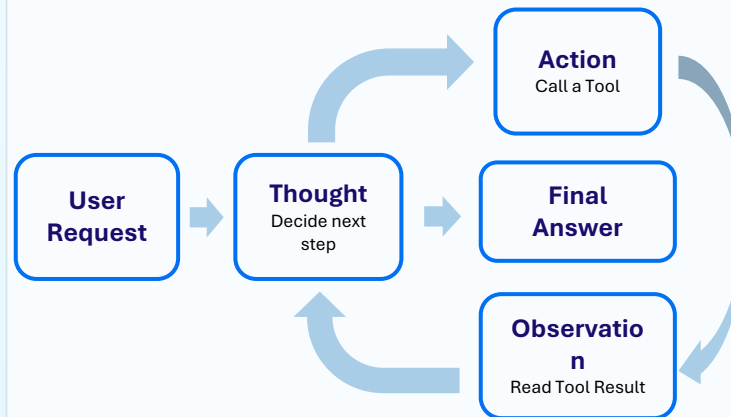
Single Prompt

One Request, one response



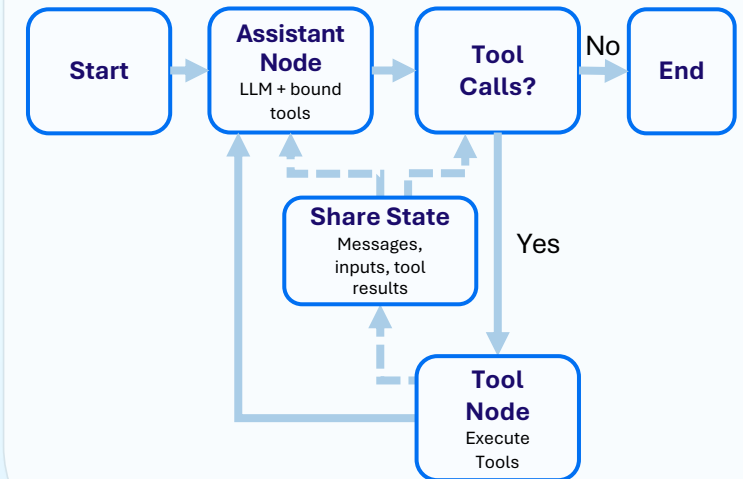
React Loop

Reason, Act, Observe, Repeat



LangGraph Workflow

Stateful Graph with explicit routes



Pre-requisites

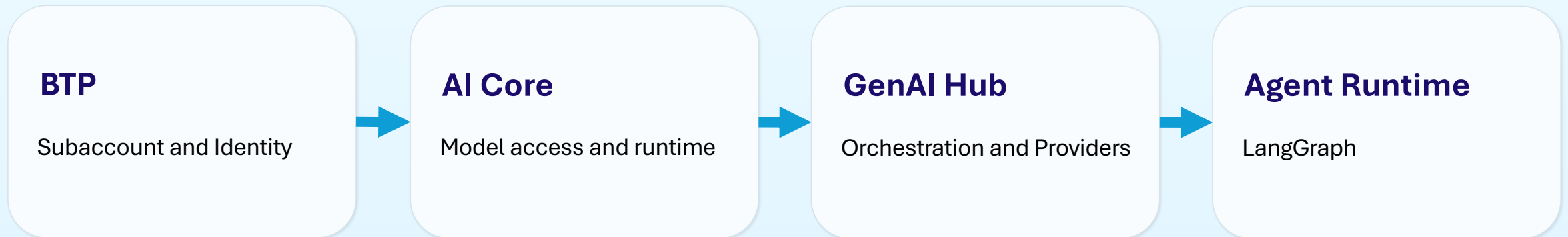
Commercial

- SAP AI Core with the Extended tier to enable Generative AI Hub
- SAP AI Launchpad for setup and operational visibility
- Optional SAP HANA Cloud when the agent queries enterprise data with SQL

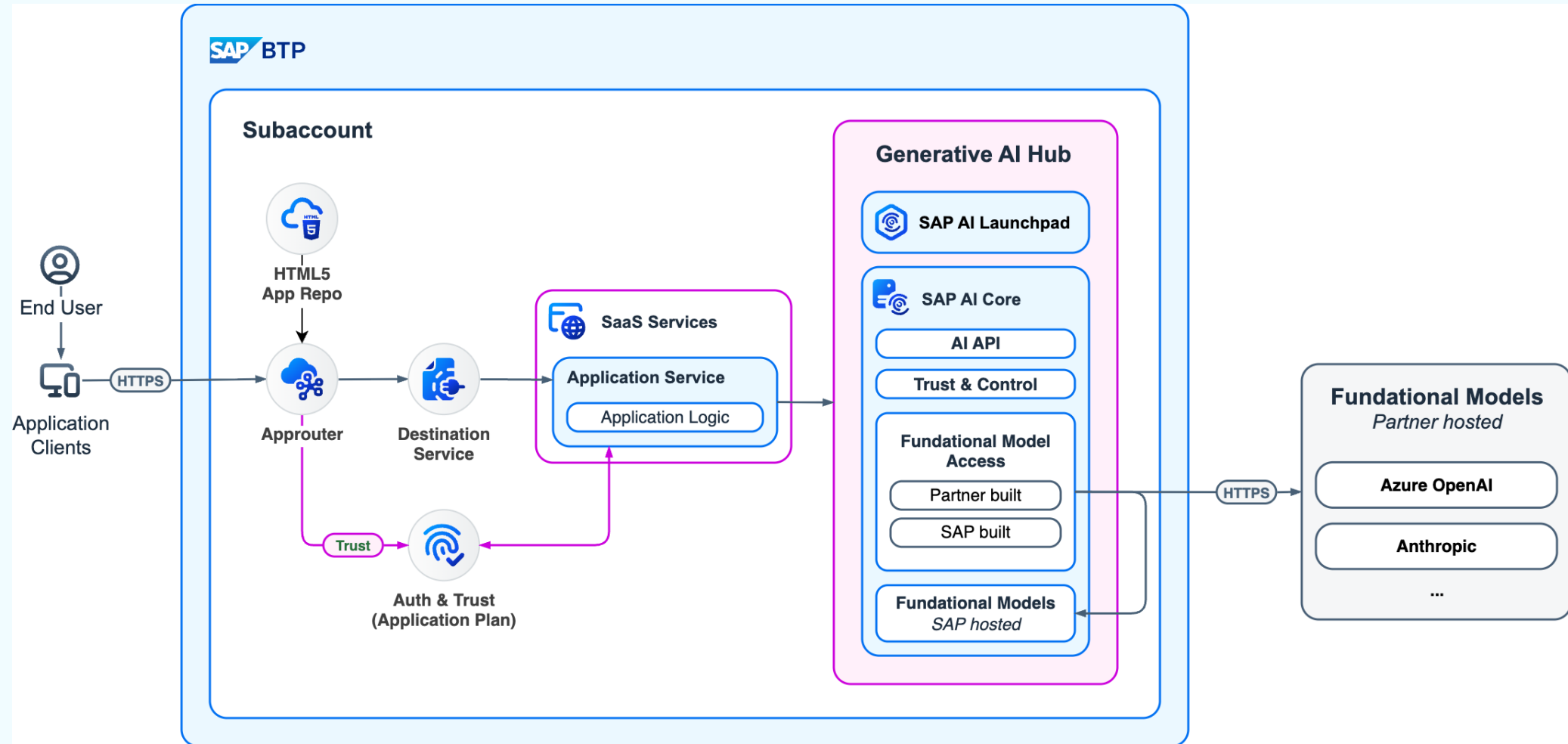
Technical

- SAP BTP subaccount, AI Core instance, and orchestration configuration
- Python 3.10+ development environment
- Key dependencies: SAP Cloud SDK for AI, LangGraph, LangChain

Minimum Stack



High-level reference architecture



Key Choices and Guidelines

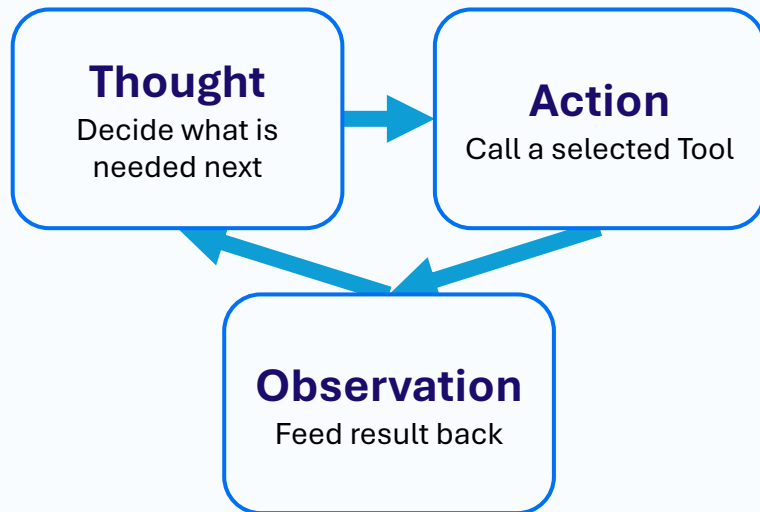
Decisions that determine reliability, control, and maintainability

1

Understand ReAct Before Frameworks

The core loop behind agentic behavior

Thought → Action → Observation



Why it matters

- Every agentic framework abstracts this same loop
- The scratchpad shows tool history and observations
- Failures are easier to debug when the loop is visible

Key Choices and Guidelines

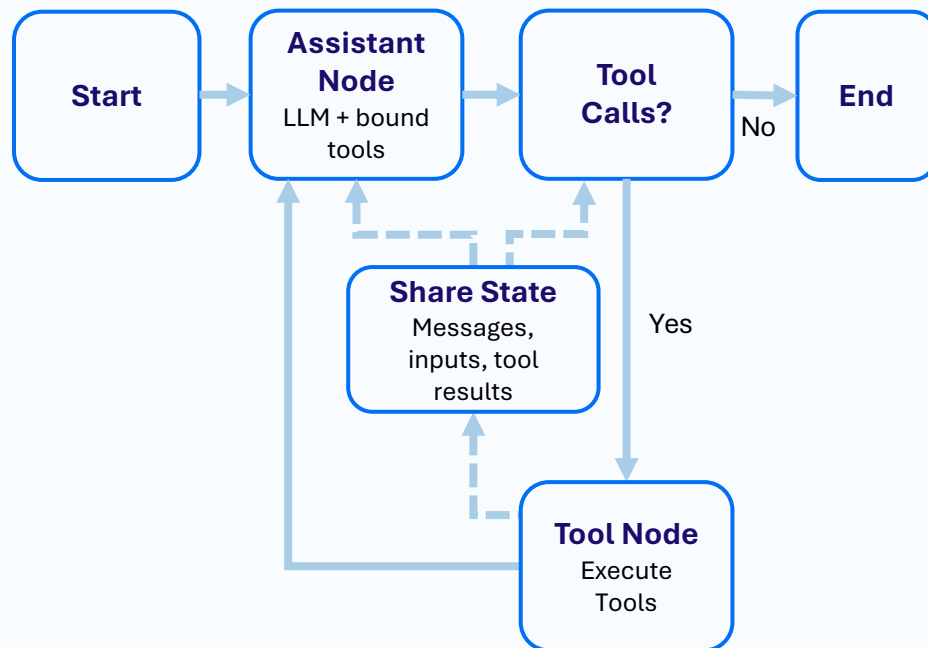
Decisions that determine reliability, control, and maintainability

2

Choose LangGraph for Production

Structured control for enterprise-grade agentic workflows

Production Graph



1

State

Typed shared state documents what data exists at each step.

2

Routing

Conditional edges make branching and fallback logic explicit.

3

Integration

LangChain wrappers connect through SAP GenAI Hub without workarounds.

Key Choices and Guidelines

Decisions that determine reliability, control, and maintainability

3

Tool Design and Scoped Access

The agent is only as reliable as the tools it can call

1

Focused

One tool has one clear responsibility. Avoid monolithic utilities that do everything.

2

Described

Names, docstrings, and parameter descriptions tell the LLM when and how to use a tool.

3

Structured

Typed inputs and concise structured outputs reduce parsing errors and long observations.

4

Scoped

Expose only the tools an agent needs. Split agents when domains or permissions differ.

Example tool set for procurement: “lookup product”, “check inventory”, “validate budget”, “check supplier constraints”, “create po draft”

Key Choices and Guidelines

Decisions that determine reliability, control, and maintainability

4

Operations, Models and Controls

Plan for reliability before the first production run

1

Model fit

Use reasoning-capable models for planning, tool selection, and structured output.

2

Determinism

Use low temperature when the agent must produce reliable tool calls.

3

Turn limits

Configure max_turns to stop runaway loops and control cost.

4

Step logging

Capture tool calls, observations, timestamps, and token usage.

5

Monitoring

Watch tool-call patterns, failures, latency, and unusual loops.

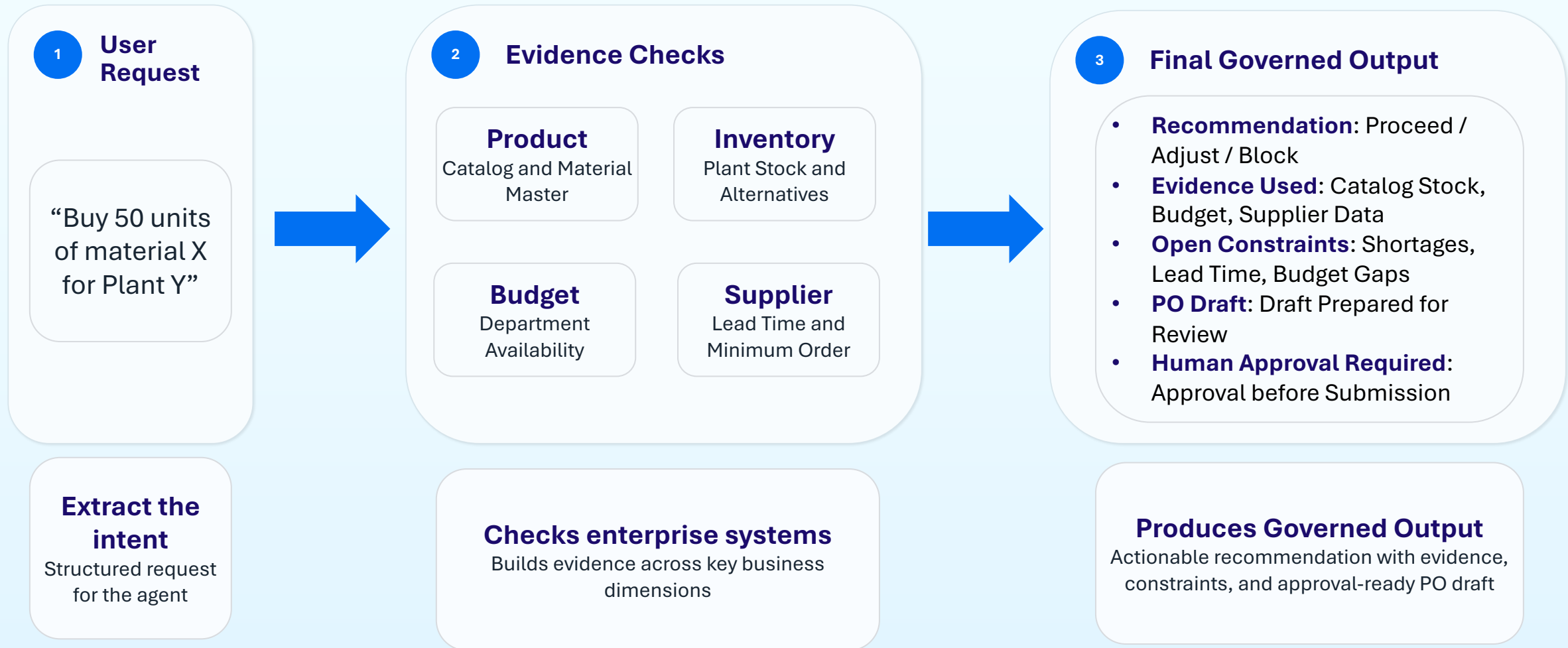
6

HITL

Require explicit approval for write, financial, or high-impact actions.

What a Final Agent Looks Like

Procurement workflow example with tools gates, and a draft output



To Know More

Adjacent Best Practices for Integration and Governance

Model Context Protocol (MCP)

- Standardizes how agents discover and invoke tools
- Reduces M x N custom integrations to a shared protocol layer
- Keeps credentials and OAuth flows outside the LLM context
- Relevant when pro-code agents need reusable enterprise tool access

Security and Governance

- Covers prompt injection, excessive agency, memory poisoning, and output handling
- Recommends least-privilege tool access, audit logging, and quotas
- Requires human approval for high-risk actions
- Frames agent deployment as defense in depth

Contributors



Robledo, Francisco

Thank you